大模型应用开发

基础学习资料

大模型基础 · 提示词工程 · 模型部署 · 微调训练 · RAG · Agent

面向人工智能相关专业学生的通用知识学习与实践参考

学习说明

本资料面向大模型应用开发的入门与进阶学习，介绍大语言模型的基本原理、常见开发技术、典型工具和应用架构。本资料是通用学习材料，不是具体竞赛任务解析。文中不提供特定赛题的操作答案、专用提示词、指定训练配置、数据处理结果或程序实现方案。学习者应通过理解原理和独立实践，掌握技术的适用场景。

建议学习顺序：先了解大模型基础和开发环境，再学习提示词与模型调用；随后学习数据处理、微调和模型评估；最后学习 RAG、Agent 和应用部署。

学习内容概览

模块	主要学习内容
一、大语言模型基础	模型概念、Transformer、Token、上下文窗口、推理生成
二、开发环境与模型部署	Python、虚拟环境、GPU、CUDA、PyTorch、模型格式、本地推理
三、提示词工程	提示词结构、角色设定、上下文组织、结构化输出、效果评估
四、模型数据与微调	数据准备、全量微调、PEFT、LoRA、QLoRA、指令微调、偏好优化
五、模型评估与优化	自动评估、人工评估、基准测试、推理参数和性能优化
六、RAG 检索增强生成	文档处理、切分、Embedding、向量数据库、混合检索、重排序
七、Agent 智能体	工具调用、任务规划、记忆、多步骤执行、多智能体
八、应用开发与部署	API、Web 应用、服务部署、日志监控、安全与合规

一、大语言模型基础

1.1 什么是大语言模型

大语言模型（Large Language Model，LLM）是基于深度学习构建的语言模型，通常通过大规模文本数据进行预训练，学习语言规律、知识关联和文本生成能力。经过指令微调等训练后，模型能够更好地理解用户意图并遵循任务要求。

- 常见能力：文本生成、问答、摘要、翻译、分类、信息抽取、代码生成和推理。
- 常见局限：可能生成不准确内容；知识可能过时；复杂任务可能出现推理错误；输出结果需要验证。

1.2 Transformer 基本结构

Transformer 是当前许多大语言模型采用的核心网络架构。其重要组成包括嵌入层、位置编码或位置信息机制、注意力模块、前馈网络、残差连接和归一化层。

组成	基本作用
Embedding	将离散 Token 映射为向量表示
位置编码/位置机制	向模型提供 Token 的位置信息
Self-Attention	建模序列中不同 Token 之间的关联
Feed Forward Network	对每个位置的表示进行非线性变换
残差连接与归一化	帮助稳定和优化深层网络训练

1.3 常见概念

概念	说明
Token	模型处理文本时使用的基本单位，不一定等于一个汉字或一个单词
Tokenizer	负责将文本切分并转换为 Token ID
上下文窗口	模型一次处理的输入与生成内容长度范围
参数量	模型可学习参数的规模，常用 B 表示十亿级参数
预训练	模型在大规模数据上学习通用语言和知识表示的阶段
指令微调	通过指令与回答样本，使模型更好地遵循用户任务
推理/Inference	使用训练完成的模型生成结果的过程

1.4 模型类型与常见系列

按模型用途，可以了解通用对话模型、代码模型、数学与推理模型、多模态模型、Embedding 模型和重排序模型等。按开放方式，可以区分开放权重模型与通过 API 提供服务的模型。

学习时应关注模型的适用任务、上下文长度、模型规模、许可证、推理资源需求和输入输出格式，而不是只比较参数量。

二、开发环境与模型部署

2.1 Python 开发基础

- 掌握 Python 基础语法、函数、类、模块、异常处理和文件读写。
- 熟悉 pip、requirements.txt、虚拟环境及依赖版本管理。
- 了解 JSON、YAML、CSV、Markdown 等常见数据格式。
- 能够使用 Git 管理代码，理解分支、提交和版本回退。

2.2 Linux 与 GPU 基础

大模型训练和部署通常在 Linux 服务器或容器中进行。需要了解目录管理、权限、进程、环境变量、日志查看、磁盘和显存监控等操作。

技术	作用与学习重点
Linux	命令行操作、文件权限、进程管理、Shell 脚本
虚拟环境	隔离项目依赖，降低不同项目之间的版本冲突
GPU	利用并行计算加速模型训练和推理
CUDA	NVIDIA GPU 计算平台及相关工具链
PyTorch	张量计算、自动求导、神经网络构建与训练
Docker	将应用及其运行依赖封装为容器，提升环境可复现性

2.3 模型部署方式

方式	特点	适用场景
云端模型 API	无需自行管理模型权重和 GPU，按接口调用	快速开发应用、验证产品方案
本地模型部署	模型运行在本地或自有服务器，数据和服务更可控	离线环境、隐私要求较高的场景
容器化部署	将模型服务及依赖封装并统一管理	团队协作、测试和生产部署

推理服务框架	提供模型加载、批处理、并发和 API 服务能力	高并发或需要优化吞吐的场景

2.4 常见模型格式与推理工具

名称	用途或特点
Safetensors	常见的模型权重存储格式
PyTorch Checkpoint	PyTorch 常用权重或训练状态保存形式
GGUF	常用于本地 CPU/GPU 混合推理生态的模型格式
Transformers	模型加载、Tokenizer、训练和推理的常用 Python 库
Ollama	简化本地大语言模型下载、管理和运行
vLLM	面向高吞吐推理服务的开源框架
llama.cpp	支持多种硬件环境的轻量化本地推理项目

学习提示：不同模型格式和推理框架并非完全互换。选择时需要检查模型架构支持、量化方式、硬件兼容性和工具版本。

三、提示词工程

3.1 提示词工程的作用

提示词工程（Prompt Engineering）是通过组织指令、上下文、示例和输出约束，引导模型完成目标任务的方法。提示词能够改善模型对任务的理解，但不能保证模型始终正确，也不能替代模型训练或外部知识检索。

3.2 常见提示词组成

组成	作用
角色与身份	说明模型需要扮演的角色或专业视角
任务目标	明确需要完成的具体任务
背景上下文	提供必要的业务背景、定义或参考材料
输入内容	说明用户提供的数据或问题
约束条件	规定边界、禁止事项和处理原则
输出格式	指定自然语言、列表、表格或 JSON 等形式
示例	通过输入与期望输出示范任务要求

3.3 常见提示词技术

- Zero-shot：不提供示例，直接描述任务。
- Few-shot：提供少量示例，帮助模型理解任务模式。
- 结构化提示：将角色、目标、上下文和约束分区组织。
- 思维引导：要求模型进行必要的分析、检查或分步骤处理；不应把模型自述过程当作真实推理证据。
- 输出约束：要求符合 JSON Schema、固定字段或指定格式，并在程序侧进行校验。
- 提示词迭代：通过测试样例观察错误类型，针对性修改指令。

3.4 提示词设计原则

- 任务描述具体，避免含糊或互相矛盾的指令。
- 提供完成任务所需的信息，不堆砌无关背景。
- 明确模型不知道或无法确认时应如何回答。

- 复杂任务拆分为可检查的步骤。
- 通过多种输入样例测试稳定性，而不是只测试一个成功案例。

四、模型数据与微调训练

4.1 为什么需要微调

当通用模型在特定领域、表达风格、任务格式或专业流程上的表现不足时，可以通过微调让模型适应目标任务。微调适合改善模型行为和任务模式，但不应被当作更新所有事实知识的唯一方式。需要频繁更新或必须可追溯的知识，通常还应考虑 RAG。

4.2 微调前的数据准备

- 明确任务目标：分类、问答、摘要、抽取、对话或代码生成。
- 检查数据来源、授权情况、隐私信息和使用范围。
- 清理乱码、重复样本、无效内容和格式错误。
- 统一字段、角色格式、编码和文本规范。
- 检查样本覆盖范围、类别分布、答案质量和潜在偏差。
- 划分训练集、验证集和测试集，避免相同或高度相似样本跨集合泄漏。

4.3 常见微调方法

方法	核心思路	优点	局限/注意点
全量微调（Full Fine-tuning）	更新模型全部或绝大多数参数	适应能力强，调整空间大	显存、算力和存储开销较高
冻结部分层微调	冻结部分网络层，只训练选定层或模块	降低训练成本	可调整参数受限，效果依赖任务
LoRA	在部分线性层加入低秩可训练矩阵，原模型主体通常冻结	参数量较少，训练和保存成本较低	需要选择适合的目标模块和训练配置
QLoRA	结合量化基座模型与 LoRA 适配器进行训练	进一步降低显存占用	量化和计算精度带来额外配置与兼容性问题
AdaLoRA	动态分配不同模块的低秩预算	可将参数预算分配给更重要的模块	配置和训练策略相对复杂
DoRA	将权重方向与幅值进行分解后进行适配	提供不同于标准 LoRA 的参数适配方式	具体效果需按模型和任务验证
Prefix Tuning	学习一组可训练前缀向量，引导模型注意力	训练参数较少	对模型结构和任务适配有要求
Prompt Tuning	学习输入侧的软提示向量	参数开销小	与自然语言提示词不同，效果依赖模型规模和任务
P-Tuning / P-Tuning v2	通过可训练连续提示向量调整模型行为	适用于参数高效适配场景	实现方式和适用范围需区分具体版本

4.4 常见训练范式

训练范式	说明	常见用途
持续预训练（Continued Pretraining）	继续使用领域语料训练语言建模目标	增强领域文本分布和术语适应能力
监督微调（SFT）	使用指令、输入和目标回答进行监督训练	提升指令遵循和特定任务能力
偏好优化	利用偏好数据调整模型输出倾向	改善回答风格、偏好和有用性
强化学习对齐	根据奖励信号优化模型策略	在可定义奖励或反馈的任务中进行行为

		优化

偏好优化方法可以进一步了解 RLHF、DPO、IPO、ORPO 等。它们在数据需求、优化目标和训练流程上有所区别，不能简单视为同一种算法。

4.5 微调训练中的关键概念

概念	含义
Epoch	完整遍历一次训练数据
Batch Size	一次参数更新所使用的样本批量大小
Learning Rate	控制参数更新幅度的重要超参数
Warmup	训练初期逐步增加学习率的策略
Weight Decay	常见正则化手段之一
Gradient Accumulation	累积多个小批次梯度后再更新参数
Gradient Clipping	限制梯度范数，降低梯度异常风险
Checkpoint	训练过程中保存的模型或状态
Overfitting	模型过度适应训练数据，泛化能力下降
Early Stopping	验证效果不再改善时提前停止训练

4.6 训练结果保存与推理

训练完成后，应区分基座模型权重、微调适配器权重、训练状态和最终合并模型。使用 PEFT 方法训练时，常见做法是保存适配器配置和权重，推理时再加载对应基座模型与适配器。是否合并权重取决于部署框架和实际需求。

学习提示：微调方法没有绝对最优选择。应综合数据规模、任务类型、GPU 显存、训练时间、模型结构、部署方式和效果要求进行选择。

五、模型评估与推理优化

5.1 为什么要评估模型

模型训练损失下降并不意味着实际应用效果一定变好。评估需要结合自动指标、人工检查和真实业务测试，观察准确性、完整性、稳定性、安全性和响应效率。

5.2 常见评估方法

方法/指标	主要用途	注意事项
Accuracy	分类任务中预测正确比例	类别不平衡时可能产生误导
Precision / Recall / F1	衡量分类结果的查准率、查全率和综合表现	需要明确正负类定义
ROUGE	常用于摘要和文本生成的词片段重合评估	不能完整衡量事实正确性
BLEU	常用于机器翻译和文本生成的 n-gram 精确匹配评估	对表达差异较敏感
Perplexity	衡量语言模型对文本序列的预测程度	不等同于实际问答质量
人工评估	由评审人员判断正确性、相关性、可读性和安全性	需制定统一标准，降低主观差异
基准测试	在标准数据集或任务集合上比较模型能力	基准表现不一定代表真实业务表现

5.3 推理参数

参数	影响
Temperature	控制生成随机性；较低通常更稳定，较高通常更多样
Top-p	从累计概率达到阈值的候选 Token 范围内采样
Top-k	限制每一步采样时考虑的候选 Token 数量
Max New Tokens	限制生成内容的最大长度
Stop Sequences	指定停止生成的标记或字符串
Repetition Penalty	降低重复生成倾向的一种控制方式

5.4 推理性能优化方向

- 量化：使用较低精度表示权重，降低显存和存储占用。
- 批处理：将多个请求合并处理，提高 GPU 利用率。
- KV Cache：缓存注意力计算中可复用的键和值。
- 连续批处理：根据请求到达和完成情况动态调度批次。
- 并行策略：了解数据并行、张量并行和流水线并行。
- 监控：关注首 Token 延迟、生成速度、吞吐量、显存和错误率。

六、RAG 检索增强生成

6.1 RAG 的基本思想

RAG（Retrieval-Augmented Generation）将外部知识检索与大语言模型生成结合。系统先从知识库中检索相关资料，再把检索内容作为上下文提供给模型，以增强回答的相关性和可追溯性。

```
用户问题
  ↓
查询处理
  ↓
知识库检索
  ↓
相关内容排序与筛选
  ↓
问题 + 检索上下文
  ↓
大语言模型生成
  ↓
回答与来源展示
```

6.2 RAG 核心组成

组件	作用
文档加载	从 PDF、DOCX、txt、网页、数据库等来源获取内容
文档解析与清洗	去除无效信息，保留正文结构和有用元数据
文本切分	将长文档拆成适合检索的文本块
Embedding	把文本编码为向量表示
向量数据库	存储向量及其对应文本和元数据
Retriever	根据用户查询召回相关文本
Reranker	对初步召回的候选内容重新排序

生成模型	结合检索上下文生成自然语言答案

6.3 文本切分策略

- 固定长度切分：按字符数或 Token 数切分，简单易实现。
- 递归切分：优先按段落、句子等边界切分，超长时再继续拆分。
- 结构化切分：利用标题、章节、条款和表格结构组织文本。
- 语义切分：根据文本语义变化确定切分边界。
- 重叠切分：相邻文本块保留部分重叠，减少上下文断裂。

6.4 Embedding 与向量检索

Embedding 模型将文本映射到向量空间。检索时将查询也转换成向量，再通过相似度计算找到语义相关的文本。常见相似度计算包括余弦相似度、点积和欧氏距离。

常见向量数据库或检索工具包括 FAISS、Milvus、Chroma、Qdrant、Weaviate 等。选择时要考虑数据规模、部署方式、过滤能力、持久化和并发需求。

6.5 检索优化方法

方法	说明
关键词检索	根据词项匹配召回内容，适合专有名词和精确词匹配
向量检索	根据语义向量相似度召回内容
混合检索	组合关键词和向量检索，兼顾词面与语义
元数据过滤	根据时间、类别、来源或权限过滤候选内容
Rerank	使用重排序模型提升候选文档的相关性排序
查询改写	将用户问题改写为更适合检索的表达
多查询检索	从不同角度生成多个查询并合并检索结果

6.6 RAG 评估

- 检索效果：召回内容是否包含回答问题所需的依据。
- 排序效果：最相关的证据是否排在前面。
- 生成效果：答案是否准确使用检索内容，有无无依据扩展。
- 溯源效果：答案能否对应到正确文档、章节或来源。
- 鲁棒性：面对无答案问题、歧义问题和相似干扰内容时表现如何。

七、Agent 智能体与工具调用

7.1 什么是 Agent

Agent（智能体）是一类能够围绕目标进行任务分析、步骤规划、工具调用和结果整理的应用系统。大语言模型可以作为 Agent 的决策或语言理解组件，但一个可用的 Agent 还需要工具接口、状态管理、执行控制和异常处理。

7.2 Agent 的常见组成

组成	作用
模型	理解任务、生成计划或决定下一步动作
工具	提供搜索、计算、数据库查询、文件处理或业务 API 等能力

规划器	把复杂目标拆分为若干步骤
记忆/状态	保存任务上下文、历史结果或用户偏好
执行器	调用工具并处理执行结果
控制器	判断继续执行、重试、结束或交给人工处理

7.3 工具调用（Tool Calling）

工具调用允许模型根据任务需要选择外部函数或服务。模型通常输出工具名称和参数，应用程序负责校验参数、执行工具，并将执行结果返回给模型。工具调用不是模型直接获得了系统权限，程序端必须控制可调用范围。

7.4 常见 Agent 工作模式

- 单 Agent：一个智能体负责理解和完成任务。
- 多步骤 Agent：将任务拆成多个步骤，按顺序执行并检查结果。
- 检索型 Agent：根据问题决定是否查询外部知识。
- 规划与执行分离：先形成计划，再由执行模块逐步完成。
- 多 Agent 协作：多个角色分工处理，再汇总结果。

7.5 Agent 常见框架

框架/工具	主要方向
LangChain	模型调用、工具、检索链和应用编排
LangGraph	基于图结构构建有状态、多步骤和可控制的 Agent 流程
LlamaIndex	数据连接、索引、检索和知识应用
AutoGen	多智能体对话与协作应用
CrewAI	基于角色和任务组织多 Agent 工作流
Semantic Kernel	将模型能力与函数、插件和企业应用集成

7.6 Agent 的风险控制

- 对工具参数进行类型、范围和权限校验。
- 设置最大执行步数、超时和重试次数。
- 对文件写入、外部发送、资金操作等高影响行为设置人工确认。
- 对工具输出进行校验，不将外部内容直接视为可信指令。
- 保留关键调用日志，支持问题追踪和审计。

八、大模型应用开发与部署

8.1 常见应用架构

用户 / 前端界面
↓
后端服务（FastAPI 等）
↓
业务逻辑与权限校验
↓
模型服务 / RAG 检索 / Agent 工具
↓
数据存储、日志与监控

8.2 常见开发工具

技术	常见用途
FastAPI	构建 Python 后端 API 服务
Flask	轻量级 Python Web 服务
Streamlit	快速构建数据与模型交互界面
Gradio	快速构建机器学习模型演示界面
Open WebUI	提供本地大模型交互界面
PostgreSQL	关系型数据存储
Redis	缓存、会话和任务状态管理
Git	代码版本管理
Docker Compose	编排多个容器服务

8.3 模型服务接口

应用通常通过 HTTP API 调用模型服务。开发时应理解请求方法、URL、Header、JSON 请求体、响应码、超时、流式响应和异常处理。部分模型服务采用 OpenAI 兼容接口，便于替换底层模型提供方，但具体支持能力仍需核对服务文档。

8.4 应用上线前检查

- 配置与密钥通过环境变量或安全配置管理，不写入公开代码仓库。
- 设置身份认证、访问控制和请求频率限制。
- 记录必要日志，避免记录密码、密钥和不必要的敏感数据。
- 对输入和模型输出进行必要的格式与安全检查。
- 准备异常处理、服务健康检查和资源监控。

九、数据安全、模型安全与应用伦理

- 数据合规：确认数据来源、使用授权、隐私保护和保存期限。
- 提示词注入：识别用户或外部文档中试图改变系统行为的恶意指令。
- 越权调用：工具执行必须遵循最小权限原则。
- 内容安全：对违法、有害或不适当请求进行恰当处理。
- 幻觉控制：通过检索证据、引用来源、拒答策略和人工复核降低错误风险。
- 人工监督：在医疗、法律、金融、应急等高风险场景中，模型结果不能替代专业人员和主管部门的判断。
- 可解释与可追溯：保留数据来源、模型版本、提示词版本和关键执行记录。

十、综合实践建议

10.1 推荐实践项目

实践项目	训练目标	可扩展方向
本地模型对话应用	模型部署、API 调用、界面交互	多模型切换、历史会话管理
提示词任务助手	提示词设计、结构化输出和结果校验	分类、摘要、信息抽取
小型文本微调实验	数据清洗、训练、验证和模型保存	比较不同微调方法
个人知识库问答	文档解析、Embedding、向量检索和 RAG	混合检索、重排序、引用
工具调用 Agent	函数定义、工具调用和流程控制	多步骤规划、状态持久化

综合智能应用	整合模型、检索、工具和 Web 服务	部署、监控和权限管理

10.2 学习成果自查

- 能够解释大语言模型、Transformer、Token 和上下文窗口等基本概念。
- 能够搭建 Python 开发环境，并理解 GPU、CUDA 和 PyTorch 的关系。
- 能够区分模型 API 调用和本地模型部署。
- 能够设计清晰的提示词，并通过测试样例持续改进。
- 能够说明全量微调、LoRA、QLoRA、Prompt Tuning 等方法的差异。
- 能够理解训练集、验证集和测试集的作用，并检查数据泄漏。
- 能够解释 ROUGE、BLEU、Precision、Recall、F1 等指标的基本用途。
- 能够说明 RAG 的文档处理、切分、Embedding、检索和生成流程。
- 能够区分关键词检索、向量检索、混合检索和重排序。
- 能够说明 Agent、工具调用、任务规划和多智能体协作的基本概念。
- 能够设计一个包含前端、后端、模型服务和数据存储的简单应用。
- 能够识别模型幻觉、提示词注入、隐私泄露和工具越权等风险。

参考学习资源

以下为通用官方文档或项目主页，可用于继续学习。具体版本、安装方式和 API 使用方法以各项目官方文档为准。

- PyTorch：https://pytorch.org/
- Hugging Face Transformers：https://huggingface.co/docs/transformers/
- PEFT：https://huggingface.co/docs/peft/
- TRL：https://huggingface.co/docs/trl/
- LangChain：https://python.langchain.com/
- LangGraph：https://langchain-ai.github.io/langgraph/
- LlamaIndex：https://docs.llamaindex.ai/
- FAISS：https://github.com/facebookresearch/faiss
- vLLM：https://docs.vllm.ai/
- Ollama：https://ollama.com/
- FastAPI：https://fastapi.tiangolo.com/
- Gradio：https://www.gradio.app/docs/
- Streamlit：https://docs.streamlit.io/